

Arquivo: metadata_service.py

```
import json # Usada para salvar e carregar o índice dos metadados
import threading
from pathlib import Path
from dfs.config import METADATA_FILE # Caminho do arquivo onde os metadados serão salvos


# Define a classe responsável pelos metadados do DFS
# Ela funciona como um pequeno serviço de índice
class MetadataService:
    """
    Serviço responsável pela indexação do DFS
    Ele mantém o mapeamento:
        caminho_lógico -> lista de chunks físicos
    """

    def __init__(self):
        # Define o caminho do arquivo de metadados, garantindo que a pasta exista
        self.metadata_file = METADATA_FILE
        self.metadata_file.parent.mkdir(parents=True, exist_ok=True)

        # Cria um lock para proteger o índice contra acesso concorrente
        self._lock = threading.Lock()
        self._index = self._load()

    # Carrega o índice do arquivo de metadados JSON
    def _load(self) -> dict:
        # Primeira execução: o arquivo de metadados não existe, então retorna um índice vazio
        if not self.metadata_file.exists():
            return {}

        try:
            # Retorna o conteúdo do arquivo de metadados como um dicionário Python
            return json.loads(self.metadata_file.read_text(encoding="utf-8"))
        except Exception:
            return {} # Se o arquivo estiver corrompido ou ilegível, retorna um índice vazio

    # Salva o índice atual em disco no formato JSON
    def _save(self) -> None:
        self.metadata_file.write_text(
            json.dumps(self._index, indent=4, ensure_ascii=False), # Salva o índice como JSON formatado p
            encoding="utf-8",
        )

    # Registra ou atualiza um arquivo no índice
    # Chamado normalmente depois que o PUT salva todos os chunks nos nós
    def put_file(self, path: str, size: int, chunks: list[dict]) -> None:
        with self._lock:
            self._index[path] = { # Cria ou substitui a entrada do arquivo no índice
                "path": path,
                "size": size,
                "chunks": chunks, # Cada chunk é um dicionário com informações sobre onde o chunk está ar
```

```
}
self._save()

# Busca as informações de um arquivo no índice
# É usado principalmente no GET e no DELETE
def get_file(self, path: str) -> dict | None:
    with self._lock:
        return self._index.get(path)

# Remove um arquivo do índice
# Deve ser chamado depois que todos os chunks forem removidos dos nós
def delete_file(self, path: str) -> None:
    with self._lock:
        self._index.pop(path, None)
        self._save()

# Lista os arquivos conhecidos pelo índice
def list_files(self) -> list[str]:
    with self._lock:
        # Retorna os caminhos lógicos dos arquivos indexados, ordenados alfabeticamente
        return sorted(self._index.keys())

# Verifica se um arquivo já existe no índice
# Pode ser usado para evitar sobrescrita ou confirmar existência
def exists(self, path: str) -> bool:
    with self._lock:
        return path in self._index
```